Universidad Tecnológica Nacional — Facultad Regional Córdoba
Ingeniería en Sistemas de Información — Ingeniería y Calidad de Software
Comisión 4K2 — Grupo 2 — Año 2026

Trabajo Práctico 6

Desarrollo Conducido por Pruebas (TDD) — Decisiones de diseño

> **User Story: Inscribirme a actividad** (5 puntos)
> COMO visitante QUIERO inscribirme a una actividad PARA reservar mi lugar en la misma.
> Caso de estudio: EcoHarmony Park.

Apellidos	Nombres	Legajo	Apellidos	Nombres	Legajo
Maldonado	Gustavo Emiliano	93370	Cruz	María Victoria	407549
Quispe Ricra	Junior Jesus	97139	Chaui	Camilo	95148
D'Andrea Escoda	Thiago Augusto	87381	Soria	Mikaela Carolina	84829
Pons	Katerinne	91093	Gastiaburú	Lucas Nicolas	74907
Rolando Corbalan	Javier Efrain	401363	Torti	Mauricio Ivan	400619
Giannantonio	Fabrizio Giael	401422	Bianchi	Mateo Leonardo	79088
Piaggio	Uriel Agustin	87599	Morales	Juan Cruz	87097
Melkón	Emanuel Sebastián	401423			

Índice

1. Introducción y alcance
2. Stack tecnológico
3. Arquitectura
4. Aplicación del ciclo Red-Green-Refactor
5. Decisiones de diseño
6. Estrategia de pruebas y cobertura
7. Trazabilidad entre pruebas de usuario y tests
8. Ejecución del producto
9. Limitaciones y mejoras futuras

1. Introducción y alcance

Este documento justifica las decisiones de diseño tomadas al implementar la User Story *Inscribirme a actividad* aplicando Desarrollo Conducido por Pruebas. El código fuente está en `04_Producto/Codigo_fuente/Ecoharmony_park/` y su `readme.md` explica cómo ejecutarlo.

El enunciado no especifica varias reglas necesarias para implementar y probar la funcionalidad: cupos, horarios, días de apertura del parque, talles y validaciones. El grupo las definió en el documento `tp6_aclaraciones_us.md` (reglas RN1 a RN7), ubicado junto a este informe. Cada regla indica si surge del enunciado o fue definida por el grupo, y todas las referencias a "RN" de este documento remiten a él.

Dentro del alcance: los seis criterios de aceptación de la US (elegir actividad con cupo, elegir horario disponible, indicar la cantidad de personas, cargar los datos de cada participante, aceptar los términos y condiciones, y recibir un correo con código QR).

Fuera del alcance: registro e inicio de sesión propios (se usa un usuario de prueba), alta de actividades y horarios desde la aplicación (se gestionan desde el panel de administración de Django), edad mínima por actividad, autocompletado de datos del titular y consulta de "mis actividades".

2. Stack tecnológico

Componente	Tecnología	Motivo
Lenguaje y framework	Python 3.13, Django 6.1	Conocimiento previo del grupo, ORM con migraciones y un framework de tests integrado con distintos niveles (`SimpleTestCase`, `TestCase`, `TransactionTestCase`).
API	Django REST Framework 3.18	Serializers para validar la entrada, excepciones con código HTTP y `APITestCase` para pruebas de punta a punta.
Base de datos	PostgreSQL 17	Soporta bloqueo de filas (`SELECT ... FOR UPDATE`), necesario para evitar la sobreventa de cupos (sección 5.5).
Tareas en segundo plano	Celery 5.6 y Redis 7	Envío del correo sin hacer esperar al usuario, con reintentos automáticos (sección 5.7).
Front	Templates de Django, CSS propio y JavaScript con `fetch`	La interfaz consume la misma API que se prueba; no duplica reglas de negocio.
QR	segno 1.6	Genera el PNG del código QR en Python puro, sin dependencias de sistema.
Dependencias y entorno	uv y Docker Compose	Instalación reproducible desde un archivo de bloqueo (`uv.lock`) y servicios levantados con un solo comando.

3. Arquitectura

Se adoptó una **arquitectura en capas** con tres responsabilidades separadas:

- **Vista (ViewSet):** solo coordina HTTP. Valida el formato con un serializer, llama al servicio y devuelve la respuesta.
- **Servicio:** contiene las reglas de negocio. No conoce HTTP ni el ORM.
- **Repositorio:** encapsula el acceso a datos y las transacciones.

El código se organiza en tres aplicaciones de Django, una por dominio. Un servicio nunca usa el repositorio de otro dominio: lo consulta a través del servicio de ese dominio.

```
apps/
├── usuarios/        Usuario con email único (destino del correo de confirmación)
├── actividades/     Actividad, Horario, calendario del parque, comando seed
└── inscripciones/   Inscripción, Participante, disponibilidad, notificación, tarea de Celery
core/                Configuración, URLs, Celery, manejador de errores, test runner
front/               Templates, CSS y JavaScript (las aplicaciones solo exponen la API)
```

```
POST /api/inscripciones/
  InscripcionViewSet → InscripcionCreateSerializer (formato)
                     → InscripcionService (reglas de negocio)
                           ├→ HorarioService → HorarioRepository          (dominio actividades)
                           ├→ CalendarioParqueService                      (lógica pura)
                           ├→ InscripcionRepository (bloqueo, conteo, alta) (dominio propio)
                           └→ ColaNotificador → Redis → worker Celery → EmailNotificador → SMTP
```

Endpoint	Descripción
`GET /api/actividades/`	Actividades activas, con si requieren talle y sus términos y condiciones (criterio 1).
`GET /api/horarios-disponibles/?actividad_id=`	Horarios futuros, con el parque abierto y con cupo, junto con los lugares libres (criterios 1 y 2).
`POST /api/inscripciones/`	Crea la inscripción (201) o devuelve el error con formato uniforme (criterios 3 a 6).

4. Aplicación del ciclo Red-Green-Refactor

Toda la lógica de negocio se construyó a partir de tests, con el ciclo completo para cada regla:

1. **Red:** se escribe un test que falla y se verifica que falle *por la razón esperada* (por ejemplo, que se lance la excepción correcta, no cualquier error).
2. **Green:** se escribe el código mínimo que hace pasar el test, sin anticipar reglas que ningún test pide.
3. **Refactor:** con todos los tests en verde, se mejora la estructura sin cambiar el comportamiento.

4.1 Evidencia en el historial de Git

El plan de SCM del grupo admite los prefijos `feat`, `fix`, `docs` y `refactor`. Para no modificar esa convención, la fase del ciclo se indica con un sufijo: `feat: ... (red)` para el test que falla, `feat: ... (green)` para la implementación y `refactor:` para la mejora de estructura. El historial de la rama `feature/tp6-tdd-inscripcion-actividad` permite seguir cada ciclo en orden.

148	**54**	**54**	**11**
commits en la rama	commits en rojo	commits en verde	commits de refactor

```
575c892 feat: agrega test de parque abierto un martes (red)
d9025a0 feat: agrega servicio de calendario del parque (green)
7d50279 feat: agrega test de parque cerrado los lunes (red)
d8c8b0d feat: cierra el parque los lunes (green)
...
200a5b5 feat: agrega test de actividad no admitida un dia cerrado (red)
9e8cee6 feat: rechaza actividades en dias que el parque esta cerrado (green)
5ee7d04 refactor: simplifica la validacion de horario en el calendario del parque
```

4.2 Técnicas aplicadas

- **Fake it:** el primer verde puede ser una implementación trivial. En el calendario, `esta_abierto` empezó devolviendo `True`; el test del lunes cerrado obligó a escribir la lógica real en el ciclo siguiente.
- **Triangulación:** un segundo caso rompe una implementación incompleta. El ciclo "horario sin cupo" se resolvió con `ocupados >= cupo`; el ciclo "más participantes que el cupo restante" la rompió y obligó a contar el grupo.
- **Valores borde:** se probaron los límites de cada regla: completar exactamente el cupo es válido, una actividad de 18:00 a 19:00 termina justo al cierre, un horario que empieza exactamente "ahora" ya comenzó, y un DNI de 6 o 9 dígitos es inválido.
- **Refactor con red de seguridad:** por ejemplo, la extracción de `Horario.inicio` al modelo, que eliminó un cálculo duplicado en dos servicios con todos los tests en verde.

4.3 Tests que no nacieron en rojo

Algunos tests se escribieron después del código y se commitearon *sin* sufijo, para no falsear la evidencia del ciclo: las pruebas de aceptación por API de PU3 a PU6 (las reglas ya estaban implementadas en el servicio), el test de concurrencia (el bloqueo ya existía), y los tests del comando `seed` y del manejador de errores (agregados para cubrir código que no era lógica de negocio). En el caso del test de concurrencia se verificó que detecta el defecto: al quitar el bloqueo temporalmente, el test falla mostrando dos inscripciones en un horario de cupo uno.

5. Decisiones de diseño

5.1 Inyección de dependencias y dobles de prueba

Decisión: `InscripcionService` recibe por constructor sus colaboradores (repositorio, servicio de horarios, notificador, calendario y reloj). Sin argumentos usa las implementaciones reales; los tests inyectan dobles.

Alternativa descartada: que el servicio cree sus dependencias internamente, lo que obliga a testearlo contra la base de datos.

Justificación: los 16 tests del servicio corren con `SimpleTestCase`, sin base de datos y en milisegundos, lo que hace práctico el ciclo TDD. Se usaron distintos tipos de doble según el caso: *fakes* en memoria para el repositorio y el servicio de horarios, un *espía* para el notificador (verifica a quién se notificó, sin enviar correos) y el calendario real, porque es lógica pura sin efectos. El beneficio se comprobó al incorporar Celery: pasar del envío inmediato al envío en segundo plano fue cambiar **una línea** del servicio (el notificador por defecto), sin modificar ninguno de sus tests.

5.2 Validación de formato separada de las reglas de negocio

Decisión: el serializer valida el formato (tipos, nombre obligatorio, DNI de 7 u 8 dígitos, edad mayor a 0, talle dentro de los valores posibles, al menos un participante). El servicio recibe los datos ya validados y aplica las reglas que dependen del estado: horario disponible, parque abierto, términos aceptados, talle requerido por la actividad, DNI duplicado y cupo.

Alternativa descartada: que el servicio invoque al serializer. Se evita así que el servicio dependa de DRF para validar y que sus tests deban atravesar la validación de formato.

Justificación: cada capa tiene una única responsabilidad y se prueba de forma aislada (12 tests del serializer y 16 del servicio). Los mensajes de error del serializer explican cómo corregir cada dato ("El DNI tiene que tener 7 u 8 números, sin puntos."), y el front reutiliza exactamente los mismos textos.

5.3 Excepciones de negocio con código HTTP y formato de error uniforme

Decisión: cada regla de negocio tiene su excepción (por ejemplo, `CupoInsuficienteError` o `TalleRequeridoError`), que hereda de `APIException` de DRF con su código HTTP, mensaje y código estable. Un manejador en `core/exception_handler.py` les da un formato uniforme: `{"codigo", "detalle"}`, más `"errores"` campo por campo cuando falla el formato.

Justificación: el código HTTP y el mensaje viven junto a la regla, y el front maneja todos los errores con una sola lógica. Se usa **409 Conflict** cuando la solicitud está bien formada pero choca con el estado actual (cupo agotado, DNI ya inscripto) y **400** para el resto. Se aceptó como concesión que el servicio dependa de las excepciones de DRF.

5.4 Reloj inyectado

Decisión: el servicio recibe una función que devuelve la hora actual (por defecto, `timezone.now`).

Justificación: la regla "no se puede reservar un horario que ya comenzó" depende del momento de ejecución. Con el reloj real, un test pasaría a la mañana y fallaría a la tarde. Inyectando el reloj, cada test fija el instante que necesita y es determinista. Se pasa la función y no su resultado, para que el servicio consulte la hora en cada inscripción.

5.5 Concurrencia: bloqueo del horario y PostgreSQL

Problema: si dos usuarios reservan el último lugar al mismo tiempo, ambos pueden leer "queda un lugar" antes de que el otro guarde, y se supera el cupo.

Decisión: el repositorio ofrece `bloquear_horario`, que abre una transacción y bloquea la fila del horario con `SELECT ... FOR UPDATE`. El servicio valida el DNI y el cupo y crea la inscripción dentro de ese bloqueo; la segunda solicitud espera a que termine la primera y entonces ve el cupo actualizado.

Justificación: la transacción vive en el repositorio y no en el servicio, porque `transaction.atomic` requiere conexión a la base y el servicio debe poder probarse sin ella; en los tests el bloqueo se reemplaza por un contexto vacío. Se adoptó PostgreSQL porque SQLite no soporta bloqueo de filas. Un test con dos hilos simultáneos (`TransactionTestCase`) verifica que solo uno obtiene el último lugar.

5.6 Disponibilidad de horarios en una sola consulta

Decisión: la lista de horarios disponibles se calcula en `DisponibilidadService`, dentro del dominio de inscripciones, porque el cupo libre depende de los participantes inscriptos. Los participantes de todos los horarios se cuentan con una única consulta agrupada (`GROUP BY`).

Justificación: calcularlo en el dominio de actividades obligaría a usar el repositorio de inscripciones desde otro dominio. Contar horario por horario generaría una consulta por cada uno (problema N+1); los tests lo impiden con `assertNumQueries(1)`. La regla "el horario ya comenzó" usa el mismo cálculo (`Horario.inicio`) en la lista y en la inscripción, de modo que la interfaz nunca ofrece un horario que la API luego rechace.

5.7 Envío del correo en segundo plano con Celery y Redis

Decisión: al confirmar la inscripción, `ColaNotificador` encola en Redis una tarea con el identificador de la inscripción; un worker de Celery la toma y envía el correo.

Justificación: medido con el servidor SMTP real de Gmail, el usuario vio su confirmación en **243 ms**, mientras que el envío tardó **4,26 s** en el worker. Sin la cola, el usuario hubiera esperado ese tiempo, y una caída del servidor de correo le habría mostrado un error aunque su inscripción ya estuviera guardada.

Manejo de fallas: se encola el identificador y no el objeto, porque viaja como JSON y el worker relee datos actualizados. Si el envío falla por el servidor de correo o la red, la tarea se reintenta hasta 3 veces con espera creciente. Si Redis no responde al encolar, la inscripción igual se confirma y el error queda registrado. En los tests, un test runner propio ejecuta las tareas de forma inmediata, por lo que no se necesita Redis.

5.8 Correo con el código QR embebido

Decisión: el correo replica el "pase" de la interfaz y muestra el QR dentro del cuerpo, como imagen *inline* referenciada por `cid:qr-inscripcion`. Incluye una versión en texto plano.

Justificación: Gmail bloquea las imágenes incrustadas en base64, mientras que las imágenes inline con `Content-ID` se muestran correctamente. Se usaron estilos inline y diseño con tablas porque los clientes de correo ignoran gran parte del CSS moderno.

5.9 Front con templates de Django que consumen la API

Decisión: las páginas son templates de Django, separadas en la carpeta `front/`, y el JavaScript consume la API con `fetch`. Las aplicaciones de Django solo contienen la API.

Justificación: existe una única implementación de las reglas de negocio, que es la que cubren los tests. El front valida los datos antes de enviarlos, con los mismos mensajes que la API, para dar respuesta inmediata; la API los vuelve a validar, porque la validación en el navegador es una comodidad y no un control de seguridad. La interfaz agrupa los horarios por día (en lugar de una lista de más de doscientos), muestra los lugares libres y marca cada error debajo del campo correspondiente.

5.10 Entorno con Docker Compose

Decisión: cuatro servicios: base de datos, Redis, API y worker. La API y el worker comparten imagen y configuración. Los contenedores corren con un usuario sin privilegios de administrador.

Justificación: cualquier integrante levanta el entorno completo con un comando. Se usa el servidor de desarrollo de Django porque, con `DEBUG` activo, sirve los archivos estáticos sin infraestructura adicional; es una decisión consciente para el alcance del trabajo (ver sección 9).

6. Estrategia de pruebas y cobertura

Las pruebas se organizan por niveles: muchas pruebas rápidas y aisladas sobre las reglas de negocio, y menos pruebas de punta a punta que verifican la integración.

Nivel	Tipo de test	Qué verifica	Tests
Reglas de negocio	`SimpleTestCase` con dobles	Servicio de inscripción, disponibilidad, calendario, serializer, encolado y manejador de errores	47
Acceso a datos	`TestCase` sobre PostgreSQL	Repositorios: transacción, conteos en una consulta, carga de relaciones	12
Correo y tareas	`TestCase`	Contenido del correo, QR embebido, fallas de envío, reintentos de Celery	6
Integración	`TestCase`	Servicio con todas sus dependencias reales	2
Concurrencia	`TransactionTestCase` con hilos	Dos inscripciones simultáneas no superan el cupo	1
Aceptación	`APITestCase`	Pruebas de usuario PU1 a PU6 y endpoints de consulta, por HTTP	11
Otros	`TestCase`	Comando `seed` y páginas del front	7
Total			**86**

6.1 Cobertura

Se midió con `coverage`, considerando sentencias y ramas, sobre `apps/` y `core/` (se excluyen migraciones y tests). La cobertura total es del **99%**, y del **100% con ramas incluidas** en todo el código con reglas de negocio: servicios, repositorios, serializers, notificadores y tarea de Celery.

Las 7 líneas sin cubrir se dejaron a propósito: métodos del panel de administración que devuelven un valor fijo, los `__str__` de los modelos y la rama de configuración del servidor de correo, que depende de variables de entorno. Testearlas no protegería ningún comportamiento real. La cobertura se usó para encontrar código sin probar, no como objetivo: una línea cubierta no garantiza que esté bien verificada.

7. Trazabilidad entre pruebas de usuario y tests

PU	Resultado	Test de aceptación (API)	Tests de la regla (servicio)
PU1	Pasa	`test_pu1_inscripcion_valida_con_talle_responde_201_y_envia_el_mail`	`test_una_solicitud_valida_crea_la_inscripcion_con_sus_participantes` `test_una_inscripcion_valida_notifica_al_usuario`

PU2	Falla	`test_pu2_horario_sin_cupo_responde_409`	`test_un_horario_sin_cupo_rechaza_la_inscripcion`
			`test_mas_participantes_que_el_cupo_restante_rechaza_la_inscripcion`
			`test_completar_exactamente_el_cupo_restante_es_valido`

PU3	Pasa	`test_pu3_sin_talle_en_una_actividad_que_no_lo_requiere_responde_201`	`test_sin_talle_en_una_actividad_que_no_lo_requiere_es_valido`

PU4	Falla	`test_pu4_horario_con_el_parque_cerrado_responde_400`	`test_un_horario_con_el_parque_cerrado_rechaza_la_inscripcion`
			`test_un_horario_inexistente_rechaza_la_inscripcion`
			`test_un_horario_de_otra_actividad_rechaza_la_inscripcion`
			`test_una_actividad_inactiva_rechaza_la_inscripcion`
			`test_un_horario_que_ya_comenzo_rechaza_la_inscripcion`

PU5	Falla	`test_pu5_sin_aceptar_los_terminos_responde_400`	`test_sin_aceptar_los_terminos_y_condiciones_rechaza_la_inscripcion`
PU6	Falla	`test_pu6_sin_talle_en_una_actividad_que_lo_requiere_responde_400`	`test_sin_talle_en_una_actividad_que_lo_requiere_rechaza_la_inscripcion`

Las reglas definidas por el grupo que no corresponden a una prueba de usuario también tienen tests: DNI duplicado en el horario (RN5, tres tests en el servicio), validaciones de formato de los participantes (RN5, serializer) y días y horarios de apertura del parque (RN3, nueve tests del calendario). Además, PU1 y PU6 se verificaron manualmente en el navegador, y la recepción del correo con el QR se comprobó en una casilla real de Gmail.

8. Ejecución del producto

El procedimiento completo está en el `readme.md` del proyecto. En resumen, desde `04_Producto/Codigo_fuente/Ecoharmony_park/`:

```
cp .env.example .env                      # completar las variables
docker compose up -d --build              # base, Redis, API y worker
docker compose exec api python manage.py seed --email tu_mail@gmail.com
docker compose exec api python manage.py test
```

La aplicación queda disponible en `http://localhost:8000` (puerto configurable). El usuario de prueba es `visitante`, con contraseña `visitante1234`.

9. Limitaciones y mejoras futuras

- **Entorno de desarrollo:** se usa el servidor de desarrollo de Django con `DEBUG` activo. Un despliegue en producción requeriría un servidor de aplicaciones (por ejemplo, Gunicorn) y un mecanismo para servir archivos estáticos.
- **Correos no encolados:** si Redis no está disponible, el correo queda registrado en el log pero no se reenvía automáticamente. Se podría guardar el estado del envío en la inscripción y reintentar los pendientes.
- **Pruebas del JavaScript:** el comportamiento del front se verificó manualmente; automatizarlo requeriría incorporar una herramienta de pruebas de navegador.
- **Funcionalidad fuera de alcance:** registro de usuarios, edad mínima por actividad, autocompletado de datos del titular y consulta de las inscripciones propias.